

Отчёт по задаче «EFF_k(alpha)»

Смола Артём, группа 23.Б08

1 Постановка задачи

Для КС-грамматики, параметра $k \geq 1$ и цепочки α требуется построить множество $EFF_k(\alpha)$:

1. Рассмотреть терминальные цепочки, получаемые правосторонними выводами из α .
2. Для каждой цепочки взять всю цепочку при длине меньше k , иначе префикс длины k .
3. Добавить результат в $EFF_k(\alpha)$ только если последнее правило вывода не является ε -продукцией.

2 Принцип решения

Реализация строит правосторонние выводы из α . Состояние — текущая сентенциальная форма. На каждом шаге раскрывается только правый нетерминал.

Для каждого терминального листа:

- добавляется префикс в множество `ALL_PREFIXES`;
- в `EFF` добавляется только если последнее правило имеет непустую правую часть.

Для защиты от бесконечного роста вывода используются ограничения `max_nodes` и `max_form_chars`.

3 Корректность

На каждом шаге раскрывается именно правый нетерминал, поэтому рассматриваются только правосторонние выводы. Для каждого терминального листа формируется префикс длины $\min(|w|, k)$, что совпадает с определением EFF_k . Фильтр по непустой правой части последнего правила точно реализует условие “последнее правило не ε -продукция”. Следовательно, множество `EFF` в программе совпадает с требуемым $EFF_k(\alpha)$.

4 Ограничения реализации

- для защиты от бесконечных выводов используются лимиты на число узлов и длину формы;

- при срабатывании лимитов печатается предупреждение, а вывод может быть неполным;
- в расширенном формате символы должны входить в объявленные множества нетерминалов и терминалов.

5 Исходный код

Исходный код программы и тестовые данные доступны в репозитории GitHub: https://github.com/artem-smola/formal_lang.

6 Код программы

Код программы:

```
#include <algorithm>
#include <cctype>
#include <iostream>
#include <set>
#include <sstream>
#include <string>
#include <string_view>
#include <unordered_map>
#include <unordered_set>
#include <utility>
#include <vector>

namespace {

struct Production {
    int id = 0;
    char lhs = 0;
    std::string rhs;
};

struct Config {
    int k = 1;
    int max_nodes = 300000;
    int max_form_chars = 256;
    bool extended_input = false;

    char start_symbol = 0;
    std::unordered_set<char> vn_decl;
    std::unordered_set<char> vt_decl;

    std::vector<Production> productions;
    std::vector<std::string> alphas;
};

struct State {
    std::string form;
    bool last_rule_non_epsilon = false;
};

std::string trim(std::string s) {
    while (!s.empty() && std::isspace(static_cast<unsigned char>(s.front()))) {
        s.erase(s.begin());
    }
}
```

```

    }
    while (!s.empty() && std::isspace(static_cast<unsigned char>(s.back()))) {
        s.pop_back();
    }
    return s;
}

bool parse_int(const std::string& token, int& value) {
    std::istringstream in(token);
    in >> value;
    return static_cast<bool>(in) && in.eof();
}

bool is_nonterminal(char c) {
    return c >= 'A' && c <= 'Z';
}

bool iequals(std::string_view a, std::string_view b) {
    if (a.size() != b.size()) {
        return false;
    }
    for (std::size_t i = 0; i < a.size(); ++i) {
        if (std::tolower(static_cast<unsigned char>(a[i])) !=
            std::tolower(static_cast<unsigned char>(b[i]))) {
            return false;
        }
    }
    return true;
}

bool is_eps_token(std::string_view s) {
    const std::string t = trim(std::string(s));
    return t.empty() || iequals(t, "eps") || iequals(t, "epsilon") || t == "ε";
}

std::vector<std::string> split_by_bar(const std::string& s) {
    std::vector<std::string> parts;
    std::string cur;
    for (char ch : s) {
        if (ch == '|') {
            parts.push_back(trim(cur));
            cur.clear();
        } else {
            cur.push_back(ch);
        }
    }
    parts.push_back(trim(cur));
    return parts;
}

void split_by_semicolons(std::vector<std::string>& out, const std::string& raw)
↪ {
    std::string x = trim(raw);
    if (x.empty()) {
        return;
    }
    std::size_t p = 0;
    while (p < x.size()) {
        std::size_t q = x.find(';', p);
        if (q == std::string::npos) {

```

```

        std::string s = trim(x.substr(p));
        if (!s.empty()) {
            out.push_back(std::move(s));
        }
        break;
    }
    std::string s = trim(x.substr(p, q - p));
    if (!s.empty()) {
        out.push_back(std::move(s));
    }
    p = q + 1;
}
}

std::vector<char> parse_symbol_tokens(const std::string& s, bool& ok,
↪ std::string& error) {
    std::vector<char> out;
    std::istringstream is(s);
    std::string tok;
    while (is >> tok) {
        if (tok.size() != 1) {
            ok = false;
            error = "Каждый символ должен быть отдельным токеном: `" + s + "`.";
            return {};
        }
        out.push_back(tok[0]);
    }
    ok = true;
    return out;
}

std::string parse_alpha_payload(const std::string& payload, bool& ok,
↪ std::string& error) {
    const std::string t = trim(payload);
    if (is_eps_token(t)) {
        ok = true;
        return "";
    }
    if (t.empty()) {
        ok = false;
        error = "Пустая alpha без eps/epsilon.";
        return {};
    }
    std::vector<char> chars;
    bool token_mode_ok = false;
    std::string token_mode_error;
    chars = parse_symbol_tokens(t, token_mode_ok, token_mode_error);
    if (token_mode_ok) {
        std::string alpha;
        alpha.reserve(chars.size());
        for (char c : chars) {
            alpha.push_back(c);
        }
        ok = true;
        return alpha;
    }
}

// Legacy-режим: допускаем слитную запись alpha без пробелов.
std::string alpha;
alpha.reserve(t.size());

```

```

    for (char c : t) {
        if (!std::isspace(static_cast<unsigned char>(c))) {
            alpha.push_back(c);
        }
    }
    if (alpha.empty()) {
        ok = false;
        error = token_mode_error;
        return {};
    }
    ok = true;
    return alpha;
}

std::string normalize_rhs_payload(const std::string& payload, bool& ok,
    ↪ std::string& error) {
    const std::string t = trim(payload);
    if (is_eps_token(t)) {
        ok = true;
        return "";
    }

    std::vector<char> chars;
    chars = parse_symbol_tokens(t, ok, error);
    if (ok) {
        std::string rhs;
        rhs.reserve(chars.size());
        for (char c : chars) {
            rhs.push_back(c);
        }
        return rhs;
    }

    // Допускаем компактную запись без пробелов: A -> aBC
    std::string rhs_compact;
    rhs_compact.reserve(t.size());
    for (char c : t) {
        if (!std::isspace(static_cast<unsigned char>(c))) {
            rhs_compact.push_back(c);
        }
    }
    if (rhs_compact.empty()) {
        return {};
    }
    ok = true;
    return rhs_compact;
}

bool parse_production_line(const std::string& line, std::vector<Production>&
    ↪ out, std::string& error) {
    const std::size_t arrow = line.find("->");
    if (arrow == std::string::npos) {
        error = "Ожидалась продукция формата `A -> ...`.";
        return false;
    }

    const std::string lhs_raw = trim(line.substr(0, arrow));
    if (lhs_raw.size() != 1 || !is_nonterminal(lhs_raw[0])) {
        error = "Левая часть продукции должна быть одним нетерминалом A..Z.";
        return false;
    }

```

```

    }
    const char lhs = lhs_raw[0];

    const std::string rhs_raw = trim(line.substr(arrow + 2));
    const std::vector<std::string> alts = split_by_bar(rhs_raw);
    for (const std::string& alt : alts) {
        bool ok = false;
        std::string err;
        const std::string rhs = normalize_rhs_payload(alt, ok, err);
        if (!ok) {
            error = err;
            return false;
        }
        out.push_back({static_cast<int>(out.size()), lhs, rhs});
    }
    return true;
}

bool has_production_arrow(const std::string& s) {
    return s.find("->") != std::string::npos;
}

bool kw_first_equals_ci(std::string_view seg, std::string_view kw) {
    const std::string s = trim(std::string(seg));
    const std::size_t sp = s.find(' ');
    const std::string first = trim(sp == std::string::npos ? s : s.substr(0,
        ↪ sp));
    return iequals(first, kw);
}

std::string rest_after_kw(std::string_view seg, std::string_view kw) {
    std::string s = trim(std::string(seg));
    if (!kw_first_equals_ci(s, kw)) {
        return {};
    }
    const std::size_t sp = s.find(' ');
    if (sp == std::string::npos) {
        return {};
    }
    return trim(s.substr(sp + 1));
}

bool parse_positive_int_token(const std::string& token, int& out) {
    if (!parse_int(token, out)) {
        return false;
    }
    return out > 0;
}

bool parse_input(Config& cfg, std::string& error) {
    std::vector<std::string> lines;
    std::string line;
    while (std::getline(std::cin, line)) {
        const std::string t = trim(line);
        if (t.empty() || t[0] == '#') {
            continue;
        }
        lines.push_back(t);
    }
    if (lines.empty()) {

```

```

    error = "Пустой ввод.";
    return false;
}

std::vector<std::string> segments;
for (const std::string& raw : lines) {
    split_by_semicolons(segments, raw);
}
if (segments.empty()) {
    error = "Пустой ввод после удаления комментариев.";
    return false;
}

bool saw_extended_kw = false;
for (const std::string& seg : segments) {
    if (kw_first_equals_ci(seg, "start") || kw_first_equals_ci(seg,
        ↪ "nonterminals") ||
        kw_first_equals_ci(seg, "terminals")) {
        saw_extended_kw = true;
        break;
    }
}
cfg.extended_input = saw_extended_kw;

cfg productions.clear();
cfg.alphas.clear();
cfg.vn_decl.clear();
cfg.vt_decl.clear();

if (!cfg.extended_input) {
    // Legacy-формат:
    // k <число> или просто <число>
    // далее:
    //   либо p и потом p продукций (совместимость со старым форматом),
    //   либо сразу произвольный порядок: продукции, alpha, max_nodes,
    //   ↪ max_form_chars.
    std::size_t p = 0;
    {
        const std::string first = trim(segments[p++]);
        if (kw_first_equals_ci(first, "k")) {
            const std::string rest = rest_after_kw(first, "k");
            if (!parse_positive_int_token(rest, cfg.k)) {
                error = "Ожидалось `k <положительное число>`.";
                return false;
            }
        }
        else if (!parse_positive_int_token(first, cfg.k)) {
            error = "Первая строка legacy-формата: `k <число>` или просто
                ↪ `<число>`.";
            return false;
        }
    }
}

if (p < segments.size()) {
    int p_count = 0;
    if (parse_positive_int_token(trim(segments[p]), p_count)) {
        if (static_cast<std::size_t>(1 + p_count) > (segments.size() -
            ↪ p)) {
            error = "Заявленное число продукций p не совпадает с
                ↪ количеством строк.";
            return false;
        }
    }
}

```

```

    }
    ++p; // пропускаем p_count
    for (int i = 0; i < p_count; ++i) {
        if (!has_production_arrow(segments[p])) {
            error = "После p ожидаются строки продукций `A ->
                ↪ ...`.`";
            return false;
        }
        if (!parse_production_line(segments[p++], cfg.productions,
            ↪ error)) {
            return false;
        }
    }
}

}

while (p < segments.size()) {
    const std::string seg = trim(segments[p++]);
    if (seg.rfind("max_nodes ", 0) == 0) {
        const std::string val = trim(seg.substr(std::string("max_nodes
            ↪ ").size()));
        if (!parse_positive_int_token(val, cfg.max_nodes)) {
            error = "max_nodes должен быть положительным числом.";
            return false;
        }
        continue;
    }
    if (seg.rfind("max_form_chars ", 0) == 0) {
        const std::string val =
            ↪ trim(seg.substr(std::string("max_form_chars ").size()));
        if (!parse_positive_int_token(val, cfg.max_form_chars)) {
            error = "max_form_chars должен быть положительным числом.";
            return false;
        }
        continue;
    }
    if (has_production_arrow(seg)) {
        if (!parse_production_line(seg, cfg.productions, error)) {
            return false;
        }
        continue;
    }

    bool ok = false;
    std::string err;
    const std::string alpha = parse_alpha_payload(seg, ok, err);
    if (!ok) {
        error = err;
        return false;
    }
    cfg.alphas.push_back(alpha);
}

if (cfg.productions.empty()) {
    error = "Не найдено ни одной продукции.";
    return false;
}
if (cfg.alphas.empty()) {
    error = "Нужна хотя бы одна alpha-цепочка.";
    return false;
}

```



```

    }

    std::unordered_set<char> vn;
    std::unordered_set<char> vt;
    for (const auto& pr : cfg.productions) {
        vn.insert(pr.lhs);
        for (char c : pr.rhs) {
            if (is_nonterminal(c)) {
                vn.insert(c);
            } else {
                vt.insert(c);
            }
        }
    }
    cfg.vn_decl = std::move(vn);
    cfg.vt_decl = std::move(vt);
    return true;
}

// Extended-формат.
bool have_k = false;
bool have_start = false;
bool have_vn = false;
bool have_vt = false;
for (const std::string& seg_raw : segments) {
    const std::string seg = trim(seg_raw);
    if (seg.empty()) {
        continue;
    }

    if (kw_first_equals_ci(seg, "k")) {
        const std::string rest = rest_after_kw(seg, "k");
        if (!parse_positive_int_token(rest, cfg.k)) {
            error = "Ожидалось `k` <положительное число>`.";
            return false;
        }
        have_k = true;
        continue;
    }

    if (kw_first_equals_ci(seg, "start")) {
        const std::string rest = rest_after_kw(seg, "start");
        if (rest.size() != 1 || !is_nonterminal(rest[0])) {
            error = "`start` должен задавать один нетерминал A..Z.";
            return false;
        }
        cfg.start_symbol = rest[0];
        have_start = true;
        continue;
    }

    if (kw_first_equals_ci(seg, "nonterminals")) {
        const std::string rest = rest_after_kw(seg, "nonterminals");
        bool ok = false;
        std::string err;
        const std::vector<char> syms = parse_symbol_tokens(rest, ok, err);
        if (!ok) {
            error = err;
            return false;
        }
        cfg.vn_decl.clear();
        for (char c : syms) {

```

```

        if (!is_nonterminal(c)) {
            error = "B `nonterminals` допустимы только символы A..Z.";
            return false;
        }
        cfg.vn_decl.insert(c);
    }
    have_vn = true;
    continue;
}
if (kw_first_equals_ci(seg, "terminals")) {
    const std::string rest = rest_after_kw(seg, "terminals");
    bool ok = false;
    std::string err;
    const std::vector<char> syms = parse_symbol_tokens(rest, ok, err);
    if (!ok) {
        error = err;
        return false;
    }
    cfg.vt_decl.clear();
    for (char c : syms) {
        if (is_nonterminal(c)) {
            error = "B `terminals` заглавные A..Z недопустимы.";
            return false;
        }
        cfg.vt_decl.insert(c);
    }
    have_vt = true;
    continue;
}
if (kw_first_equals_ci(seg, "alpha")) {
    const std::string rest = rest_after_kw(seg, "alpha");
    bool ok = false;
    std::string err;
    const std::string alpha = parse_alpha_payload(rest, ok, err);
    if (!ok) {
        error = err;
        return false;
    }
    cfg.alphas.push_back(alpha);
    continue;
}
if (seg.rfind("max_nodes ", 0) == 0) {
    const std::string val = trim(seg.substr(std::string("max_nodes ")
        ↪ ").size()));
    if (!parse_positive_int_token(val, cfg.max_nodes)) {
        error = "max_nodes должен быть положительным числом.";
        return false;
    }
    continue;
}
if (seg.rfind("max_form_chars ", 0) == 0) {
    const std::string val = trim(seg.substr(std::string("max_form_chars ")
        ↪ ").size()));
    if (!parse_positive_int_token(val, cfg.max_form_chars)) {
        error = "max_form_chars должен быть положительным числом.";
        return false;
    }
    continue;
}
if (has_production_arrow(seg)) {

```

```

        if (!parse_production_line(seg, cfg.productions, error)) {
            return false;
        }
        continue;
    }

    error = "Неизвестный фрагмент: `" + seg + "`.";
    return false;
}

if (!have_k || !have_start || !have_vn || !have_vt) {
    error = "Для extended-формата обязательны поля: k, start, nonterminals,
    ↪ terminals.";
    return false;
}

if (!cfg.vn_decl.count(cfg.start_symbol)) {
    error = "Символ start должен принадлежать множеству nonterminals.";
    return false;
}

if (cfg.productions.empty()) {
    error = "Не найдено ни одной продукции.";
    return false;
}

if (cfg.alphas.empty()) {
    error = "Нужна хотя бы одна alpha-цепочка.";
    return false;
}

for (const auto& pr : cfg.productions) {
    if (!cfg.vn_decl.count(pr.lhs)) {
        error = std::string("Нетерминал `" + pr.lhs + "` в левой части не
        ↪ объявлен в nonterminals.");
        return false;
    }
    for (char c : pr.rhs) {
        if (is_nonterminal(c)) {
            if (!cfg.vn_decl.count(c)) {
                error = std::string("Символ `" + c + "` из RHS не объявлен
                ↪ в nonterminals.");
                return false;
            }
        } else if (!cfg.vt_decl.count(c)) {
            error = std::string("Символ `" + c + "` из RHS не объявлен в
            ↪ terminals.");
            return false;
        }
    }
}

for (const std::string& alpha : cfg.alphas) {
    for (char c : alpha) {
        if (is_nonterminal(c)) {
            if (!cfg.vn_decl.count(c)) {
                error = std::string("Символ `" + c + "` из alpha не
                ↪ объявлен в nonterminals.");
                return false;
            }
        } else if (!cfg.vt_decl.count(c)) {
            error = std::string("Символ `" + c + "` из alpha не объявлен в
            ↪ terminals.");
        }
    }
}

```

```

        return false;
    }
}
return true;
}

std::size_t rightmost_nonterminal_pos(const std::string& s) {
    for (std::size_t i = s.size(); i > 0; --i) {
        if (is_nonterminal(s[i - 1])) {
            return i - 1;
        }
    }
    return std::string::npos;
}

bool all_terminals(const std::string& s) {
    for (char c : s) {
        if (is_nonterminal(c)) {
            return false;
        }
    }
    return true;
}

std::string pref_k(const std::string& s, int k) {
    if (static_cast<int>(s.size()) <= k) {
        return s;
    }
    return s.substr(0, static_cast<std::size_t>(k));
}

std::string printable_word(const std::string& s) {
    return s.empty() ? "ε" : s;
}

std::string printable_alpha(const std::string& s) {
    return s.empty() ? "epsilon" : s;
}

void print_set_line(const std::set<std::string>& words) {
    if (words.empty()) {
        std::cout << "∅\n";
        return;
    }
    bool first = true;
    for (const std::string& w : words) {
        if (!first) {
            std::cout << ' ';
        }
        std::cout << printable_word(w);
        first = false;
    }
    std::cout << '\n';
}

} // namespace

int main() {
    std::ios::sync_with_stdio(false);

```

```

std::cin.tie(nullptr);

Config cfg;
std::string error;
if (!parse_input(cfg, error)) {
    std::cerr << "Ошибка ввода: " << error << "\n";
    return 1;
}

std::unordered_map<char, std::vector<int>> by_lhs;
by_lhs.reserve(cfg productions.size());
for (int i = 0; i < static_cast<int>(cfg productions.size()); ++i) {
    by_lhs[cfg productions[i].lhs].push_back(i);
}

for (std::size_t qi = 0; qi < cfg.alphas.size(); ++qi) {
    const std::string& alpha = cfg.alphas[qi];
    std::vector<State> stack;
    stack.push_back({alpha, false});

    std::size_t nodes_created = 1;
    bool truncated = false;
    std::set<std::string> eff;
    std::set<std::string> all_prefixes;

    while (!stack.empty()) {
        State cur = std::move(stack.back());
        stack.pop_back();

        if (all_terminals(cur.form)) {
            const std::string pk = pref_k(cur.form, cfg.k);
            all_prefixes.insert(pk);
            if (cur.last_rule_non_epsilon) {
                eff.insert(pk);
            }
            continue;
        }

        const std::size_t pos = rightmost_nonterminal_pos(cur.form);
        if (pos == std::string::npos) {
            continue;
        }

        const char nt = cur.form[pos];
        auto it = by_lhs.find(nt);
        if (it == by_lhs.end()) {
            continue;
        }

        for (int prod_id : it->second) {
            const Production& p = cfg productions[prod_id];
            std::string next;
            next.reserve(cur.form.size() + p.rhs.size());
            next.append(cur.form.begin(), cur.form.begin() +
                ↪ static_cast<std::ptrdiff_t>(pos));
            next += p.rhs;
            next.append(cur.form.begin() + static_cast<std::ptrdiff_t>(pos +
                ↪ 1), cur.form.end());

            if (static_cast<int>(next.size()) > cfg.max_form_chars) {

```

```

        truncated = true;
        continue;
    }
    ++nodes_created;
    if (nodes_created > static_cast<std::size_t>(cfg.max_nodes)) {
        truncated = true;
        continue;
    }

    stack.push_back({std::move(next), !p.rhs.empty()});
}

}

if (cfg.alphas.size() > 1) {
    std::cout << "ALPHA " << (qi + 1) << ": " << printable_alpha(alpha)
        << "\n";
}
std::cout << "EFF:\n";
print_set_line(eff);
std::cout << "ALL_PREFIXES:\n";
print_set_line(all_prefixes);
if (truncated) {
    std::cout << "WARNING: вывод обрезан лимитами
        << max_nodes/max_form_chars.\n";
}
if (qi + 1 != cfg.alphas.size()) {
    std::cout << "\n";
}
}

return 0;
}

```

7 Демонстрация работы программы

1) Входные данные

Рассматриваемый тест: **test1**.

Формат входных данных:

1. Поддерживаются краткий и расширенный режимы ввода.
2. В кратком режиме задаётся k , затем продукции и цепочка α .
3. В расширенном режиме используются директивы **k**, **start**, **nonterminals**, **terminals**, строки **alpha** и продукции.
4. В обоих режимах можно задать **max_nodes** и **max_form_chars**.

Нетерминалы — символы $A..Z$. Пустая альтернатива, **eps** или **epsilon** задают ϵ -продукцию.

```

2
2
S -> aA | b
A -> c | eps
S

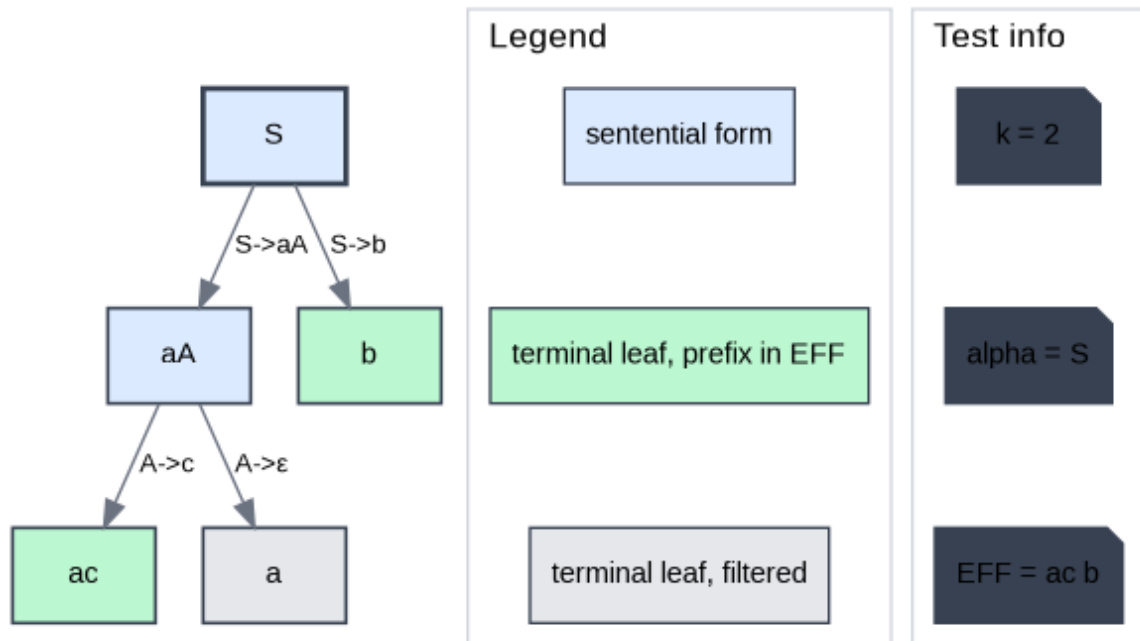
```

2) Вывод

```
• artem@artem-BMH-WCX9:~/ТП_6_семестр/ТЮЯиТ/Задания/EFF/Solution/Code$ ./eff < tests/test1.in
EFF:
ac b
ALL_PREFIXES:
a ac b
```

3) Визуализация

EFF rightmost derivations: test1



4) Описание результата

Для теста **test1** получено:

- $EFF = \{ac, b\}$;
- $ALL_PREFIXES = \{a, ac, b\}$.

Цепочка **a** присутствует в **ALL_PREFIXES**, но отсутствует в **EFF**. Причина в том, что она получена веткой, где последнее применённое правило — ϵ -продукция. Этот пример показывает, почему **ALL_PREFIXES** может быть строго больше **EFF**.

5) Дополнительный граничный тест

Для **test3** получается пустое слово в **ALL_PREFIXES**:

Входные данные теста **test3**:

```
2
3
S -> AB | c
A -> eps
B -> eps | d
S
```

$$\text{ALL_PREFIXES} = \{\varepsilon, c, d\}, \quad \text{EFF} = \{c\}.$$

Это демонстрирует фильтрацию финальной ε -ветви и корректную обработку пустой цепочки.

8 Анализ сложности

Пусть:

- M — число реально раскрытых сентенциальных форм;
- R — среднее число продукций для раскрываемого нетерминала;
- L — средняя длина формы.

Тогда время:

$$O(M \cdot R \cdot L),$$

так как каждый переход строит новую форму заменой правого нетерминала.

Память:

$$O(M \cdot L),$$

на хранимые формы в стеке и в множестве уже раскрытых состояний.